

Day 16 – Model Training, Fine-Tuning, PEFT & Evaluation

1. The big picture

A useful mental model is:

- **Pre-training teaches the model language and general knowledge.**
- **Fine-tuning teaches the model how to behave for a narrower purpose.**
- **RAG gives the model external information at inference time.**
- **Prompt engineering tells the model what to do in the current request.**

A Senior AI Engineer should not immediately suggest fine-tuning. The first question is:

Is the problem caused by missing knowledge, inconsistent behavior, weak task performance, or an unsuitable base model?

Problem	Usually start with
Model lacks current company information	RAG
Output format is inconsistent	Prompting, structured output, validation
Model does not follow domain-specific instructions reliably	Fine-tuning
Model must reproduce a consistent tone or style	Fine-tuning
Model lacks vocabulary or patterns from a specialized domain	Domain adaptation
Latency or cost is too high	Smaller model, quantization, distillation
Model occasionally hallucinates	Grounding, RAG, verification—not fine-tuning alone

2. Training vs fine-tuning vs PEFT

2.1 Pre-training

Pre-training means training a model, usually from random or nearly random weights, on a massive dataset.

For a decoder-only LLM, the common objective is next-token prediction:

Input:
The customer requested a refund because

Target:
the

The model learns to predict the next token repeatedly across billions or trillions of tokens.

During pre-training, the model learns:

- Grammar and language structure
- General facts and concepts
- Reasoning patterns

- Programming syntax
- Relationships between words and ideas

Characteristics

- Requires enormous datasets
- Requires large GPU clusters
- Takes weeks or months
- Updates all model parameters
- Is normally performed by foundation-model providers

A typical enterprise team usually does **not** train a general-purpose LLM from scratch.

2.2 Fine-tuning

Fine-tuning starts with an already trained model and continues training it on a smaller, targeted dataset.

Instead of teaching language from the beginning, fine-tuning teaches the model:

- A new task
- A preferred response format
- Domain-specific behavior
- Organization-specific terminology
- A writing style
- Tool-calling patterns

Example

The base model may answer:

You can restart the service by accessing the deployment platform.

After task-specific fine-tuning, it may answer:

1. Check the deployment status.
 2. Review the latest application logs.
 3. Restart only the affected pod.
 4. Verify health-check recovery.
 5. Escalate if the pod enters CrashLoopBackOff again.

The model has learned the organization's preferred support procedure and response style.

2.3 Full fine-tuning

In full fine-tuning, all or most model parameters are updated.

For a 7-billion-parameter model, approximately 7 billion weights may participate in optimization.

Advantages

- Maximum flexibility
- Can produce strong task adaptation
- Useful when substantial behavioral changes are required

- May outperform adapter-based methods with enough high-quality data and compute

Disadvantages

- High GPU memory requirement
- Expensive training
- Large checkpoints
- Higher risk of catastrophic forgetting
- A separate full model may be needed for each use case
- Harder to manage operationally

When full fine-tuning may be justified

- You have a large, high-quality dataset
- The task differs significantly from the base model's behavior
- You control substantial training infrastructure
- Adapter methods have been tested and are insufficient
- The value of the performance gain justifies the cost

3. Parameter-Efficient Fine-Tuning

Parameter-Efficient Fine-Tuning, or **PEFT**, updates only a small portion of the model rather than updating every parameter.

The base model remains mostly or completely frozen.

Common PEFT approaches include:

- LoRA
- QLoRA
- Adapters
- Prefix tuning
- Prompt tuning

For interviews, LoRA and QLoRA are the most important.

3.1 LoRA

LoRA means Low-Rank Adaptation.

Instead of changing the original weight matrix (W), LoRA learns a small update represented by two smaller matrices:

$$[W' = W + \Delta W]$$

where:

$$[\Delta W = BA]$$

The original model weight (W) stays frozen. Only matrices (A) and (B) are trained.

Intuition

Suppose a model contains a very large matrix:

4096×4096

Updating this full matrix requires millions of trainable values.

LoRA approximates the required change using two smaller matrices:

$$\begin{matrix} 4096 & \times & 16 \\ 16 & \times & 4096 \end{matrix}$$

The rank 16 is much smaller than 4096, so far fewer parameters need training.

Where LoRA is commonly applied

LoRA adapters are often inserted into transformer linear layers, such as:

- Query projection
- Key projection
- Value projection
- Output projection
- Feed-forward projection layers

The exact target modules should be treated as an experiment rather than selected blindly.

Advantages

- Lower GPU memory usage
- Faster training
- Smaller checkpoints
- Multiple adapters can share one base model
- Easy to switch between domain-specific adapters
- Lower operational storage cost

Limitations

- May underperform full fine-tuning for major model changes
- Rank and target-layer selection affect results
- Training only attention projections may not be enough for every task
- Adapter serving can add operational complexity

3.2 QLoRA

QLoRA combines quantization with LoRA.

The main idea is:

1. Load the base model in low precision, commonly 4-bit.
2. Keep the quantized base model frozen.
3. Add trainable LoRA adapters.
4. Train only the adapters.

$$\begin{matrix} \text{Frozen 4-bit base model} \\ + \\ \text{Trainable LoRA adapters} \\ = \\ \text{Lower-memory fine-tuning} \end{matrix}$$

Why QLoRA matters

A full-precision model may not fit on the available GPU.

QLoRA reduces the memory needed for the frozen base model, allowing larger models to be adapted using less hardware.

Important clarification

QLoRA does not normally train the 4-bit base weights directly.

The quantized model is used during forward and backward computation, while the trainable adapter parameters are maintained in a suitable training precision.

LoRA vs QLoRA

Area	LoRA	QLoRA
Base model precision	Often 16-bit or bfloat16	Usually 4-bit
Base model weights	Frozen	Frozen
Trained parameters	LoRA adapters	LoRA adapters
Memory requirement	Low	Even lower
Training complexity	Simpler	Slightly more complex
Typical use	Sufficient GPU memory available	GPU memory is highly constrained

4. Fine-tuning categories

Several different processes are often casually called “fine-tuning.” In an interview, distinguish them clearly.

4.1 Continued pre-training

Continued pre-training trains the model further using unlabeled domain text with the original language-model objective.

Example datasets:

- Medical literature
- Legal documents
- Financial reports
- Internal technical documentation
- Telecom incident logs

Purpose

It helps the model become familiar with:

- Specialized terminology
- Domain-specific language patterns
- Rare token combinations
- Industry-specific relationships

Example

Input text:
A BGP route reflector reduces the need for a full iBGP mesh...

There is no instruction-answer pair. The model continues learning by predicting tokens.

Use it when

- The model struggles to understand domain language
- A large amount of raw domain text is available
- The domain differs significantly from general internet text

Limitation

Continued pre-training does not automatically make the model a good instruction follower.

It may be followed by supervised instruction tuning.

4.2 Supervised Fine-Tuning

Supervised Fine-Tuning, or **SFT**, uses examples of the desired input-output behavior.

Example:

```
{
  "system": "You are an enterprise incident-analysis assistant.",
  "input": "The Kubernetes pod is in CrashLoopBackOff. What should I check?",
  "output": "Check the previous container logs, exit code, environment variables, mounted secrets, resource limits, and recent deployment changes."
}
```

SFT teaches the model:

- What kind of answer to produce
 - Which format to use
 - Which steps to prioritize
 - How much detail to provide
 - Which policies to follow
-

4.3 Preference tuning

Preference tuning uses comparisons between better and worse responses.

Example:

Prompt:
Explain the production incident.

Response A:
The application failed.

Response B:

The deployment introduced an invalid database configuration. Pods failed during startup, causing health checks to fail and traffic to be removed.

Preferred response:

B

The preference data may be used by methods such as:

- Reinforcement Learning from Human Feedback
- Direct Preference Optimization
- Other ranking or preference-optimization approaches

Preference tuning is especially useful for subjective qualities such as:

- Helpfulness
- Conciseness
- Tone
- Safety
- Explanation quality
- Response organization

5. Practical fine-tuning scenarios

Scenario 1: Domain adaptation

Problem

A general-purpose model frequently misunderstands:

- Internal platform terminology
- Telecom abbreviations
- Legal language
- Medical concepts
- Highly specialized technical logs

Possible design

```
Raw domain documents
    ↓
Continued pre-training
    ↓
Domain-adapted base model
    ↓
Instruction tuning
    ↓
Domain assistant
```

Senior-level decision

First determine whether the issue is truly model understanding.

If the problem is merely that the model does not know the current incident record or latest internal procedure, RAG may be more appropriate.

Good reason to fine-tune

The model repeatedly misinterprets the language itself, even when the correct context is provided.

Scenario 2: Style tuning

Problem

A customer-service assistant must always respond:

- Politely
- Briefly
- Without technical jargon
- With approved company terminology
- Using a consistent structure

Example required format:

Acknowledgement Resolution Next action Expected timeline

Prompting may initially work, but the model may inconsistently follow the format.

Fine-tuning approach

Create high-quality examples showing:

- Approved tone
- Correct length
- Proper formatting
- Phrases to avoid
- Escalation behavior

Important caution

Fine-tuning can teach communication style, but hard business rules should still be enforced using application logic.

For example, refund eligibility should be determined by a policy service—not invented by the LLM.

Scenario 3: Task-specific tuning

Problem

The model must convert unstructured incident reports into structured JSON.

Input:

Payment API latency increased after deployment v2.4. P95 reached 3.8 seconds. Rollback restored normal performance.
--

Expected output:


```
{
  "service": "Payment API",
  "incident_type": "latency",
  "metric": "p95",
  "observed_value": "3.8 seconds",
  "suspected_cause": "deployment v2.4",
  "resolution": "rollback"
}
```

Why fine-tuning may help

- Output schema is stable
- Thousands of historical labeled examples exist
- Prompt-only extraction is inconsistent
- The task is repeated at high volume
- Reducing prompt size improves cost and latency

Production design

Fine-tuning should be combined with:

- JSON schema validation
- Retry or repair logic
- Confidence checks
- Monitoring by field
- Human review for sensitive cases

Scenario 4: Tool-calling behavior

Problem

An agent frequently selects the wrong tool or sends incorrect arguments.

Available tools:

```
search_logs(service, start_time, end_time)
restart_service(service, environment)
create_incident(title, severity)
```

Fine-tuning examples can teach:

- Which tool to select
- How to extract arguments
- When not to call a tool
- When user confirmation is required
- How to recover from tool errors

However, authorization and destructive-action controls must remain in deterministic application code.

Scenario 5: Smaller specialized model

Problem

A large general-purpose model performs well but is:

- Too expensive
- Too slow
- Difficult to host privately

A smaller open model may be fine-tuned for one narrow task such as:

- Intent classification
- Log summarization
- Ticket routing
- Structured extraction
- SQL generation for a fixed schema

A fine-tuned smaller model can sometimes match or exceed a larger model on the specific task while reducing cost and latency.

6. When not to fine-tune

Fine-tuning is usually the wrong first solution when:

The information changes frequently

Examples:

- Product prices
- Current policies
- Recent incidents
- Inventory
- Employee information
- Daily financial data

Use RAG, APIs, or database tools instead.

The application has very few examples

With only 20–30 examples, prompting or few-shot prompting may be more effective and easier to maintain.

The problem is deterministic

Examples:

- Tax calculation
- Access-control decision
- Eligibility validation
- Required-field checking

Use code or a rules engine.

The prompt has not been properly optimized

Before fine-tuning, test:

- Clear system instructions
- Few-shot examples
- Structured output
- Retrieval improvements
- Better model selection
- Validation and retry logic

The base model is simply too weak

Fine-tuning cannot always compensate for:

- Poor reasoning ability
- Inadequate context window
- Weak multilingual capability
- Missing tool support
- Fundamental architecture limitations

Sometimes the correct solution is a better base model.

7. Data preparation

Fine-tuning quality is highly dependent on dataset quality.

A useful principle is:

| A small set of accurate, representative examples is often better than a much larger noisy dataset.

7.1 Instruction format

A conversational example generally contains roles such as:

```
{
  "messages": [
    {
      "role": "system",
      "content": "You are a technical support assistant. Give safe, actionable answers."
    },
    {
      "role": "user",
      "content": "My deployment is failing its readiness probe."
    },
    {
      "role": "assistant",
      "content": "Check the readiness endpoint, container port, initial delay, timeout, and application startup logs."
    }
  ]
}
```

Conceptually, this represents:

System instruction
+
User input
+
Desired assistant output

The exact serialization must match the model's expected chat template.

Incorrect special tokens or role formatting can significantly reduce training quality.

7.2 Dataset cleaning

Clean the dataset for:

- Broken or incomplete responses
- Invalid JSON
- Incorrect labels
- Contradictory instructions
- Irrelevant examples
- Encoding problems
- Excessively long samples
- Low-quality synthetic outputs
- Deprecated organizational policies

Example of a dangerous contradiction:

Example 1: Never restart production without approval.
Example 2: Restart production immediately when latency increases.

The model receives conflicting behavioral signals.

7.3 Deduplication

Duplicate examples can cause:

- Overfitting
- Memorization
- Biased evaluation
- Overrepresentation of a particular pattern
- Artificially low validation loss

Deduplication should include:

- Exact duplicate detection
- Near-duplicate detection
- Template-level duplicate detection
- Semantic similarity checks

A particularly serious problem is overlap between training and test sets.

The model may appear highly accurate because it has already seen near-identical examples.

7.4 PII removal

Personally identifiable information may include:

- Names
- Email addresses
- Phone numbers
- Account numbers
- Addresses
- Government identifiers
- Customer IDs
- Authentication tokens
- IP addresses, depending on context
- Sensitive support conversations

Possible controls include:

- Regex-based detection
- Named-entity recognition
- Data-loss-prevention systems
- Token and secret scanners
- Manual review
- Synthetic replacement

Example:

Before: Customer Rahul Sharma, account 7845120091, reported...
After: Customer <PERSON>, account <ACCOUNT_ID>, reported...

PII removal must happen before data enters uncontrolled training pipelines.

7.5 Data distribution

The training dataset should reflect production traffic.

Suppose production requests are:

Request type	Production share
Troubleshooting	50%
How-to questions	25%
Summarization	15%
Escalation	10%

If 80% of training examples are summarization tasks, the model may perform well in evaluation but poorly in actual production.

Dataset construction should account for:

- Common cases
 - Edge cases
 - Safety-sensitive requests
 - Ambiguous inputs
 - Very short inputs
 - Long inputs
 - Invalid inputs
 - Multi-turn interactions
 - Tool failures
 - Refusal examples
-

7.6 Negative examples

Do not only include perfect normal requests.

Include examples teaching the model when to:

- Ask for missing information
- Refuse unsafe actions
- Avoid guessing
- Escalate to a human
- Return “not found”
- Avoid calling a destructive tool
- Distinguish similar intents

Without negative examples, the model may become excessively confident.

7.7 Synthetic data

Synthetic data is generated by another model rather than directly written or labeled by humans.

Benefits

- Faster dataset creation
- Better coverage of rare scenarios
- Easier format generation
- Lower labeling cost

Risks

- Hallucinated labels
- Repetitive writing style
- Hidden model bias
- Low diversity
- Incorrect domain assumptions
- Teaching the student model the teacher model’s mistakes

A strong process is:

```
Seed examples
  ↓
Synthetic generation
  ↓
Rule-based validation
  ↓
Domain-expert review
  ↓
Deduplication
  ↓
Training set
```

8. Dataset splitting

Use separate datasets for:

- Training
- Validation
- Testing

Training set

Used to update model parameters.

Validation set

Used during experimentation for:

- Hyperparameter selection
- Early stopping
- Model comparison
- Detecting overfitting

Test set

Used only for final evaluation.

Important split strategy

Do not always split randomly by row.

For example, if one customer incident produces 15 similar conversations, placing 12 in training and 3 in testing causes leakage.

Better grouping units may include:

- Customer
- Incident
- Document
- Time period
- Product
- Conversation thread
- Template family

A time-based test set is valuable when production data evolves over time.

9. Fine-tuning workflow

A production-oriented workflow is:

1. Define the business problem
2. Establish the baseline
3. Choose the base model
4. Create a representative evaluation set
5. Collect and clean training data
6. Select full fine-tuning or PEFT
7. Run a small experiment
8. Evaluate quality, safety, cost, and latency
9. Perform error analysis
10. Iterate on data and hyperparameters
11. Run human evaluation
12. Deploy gradually
13. Monitor production behavior

Always establish the baseline first

Compare the fine-tuned model against:

- Base model with zero-shot prompting
- Base model with few-shot prompting
- Base model with RAG
- Stronger hosted model
- Existing rules or classifier
- Previous production version

Without a baseline, “the fine-tuned model looks good” is not meaningful.

10. Key training choices

Learning rate

A learning rate that is too high can:

- Destroy useful pretrained behavior
- Cause unstable training
- Increase forgetting

A learning rate that is too low may:

- Produce very little adaptation
 - Require more steps
 - Underfit the task
-

Number of epochs

Too few epochs:

- Underfitting
- Insufficient task learning

Too many epochs:

- Memorization
- Reduced generalization
- Catastrophic forgetting
- Overconfident outputs

Validation metrics should be monitored across checkpoints.

Batch size

Larger batches can produce stable gradient estimates but require more memory.

Gradient accumulation can simulate a larger effective batch:

Effective batch size
= micro-batch size
× gradient accumulation steps
× number of devices

LoRA rank

LoRA rank controls adapter capacity.

- Lower rank: smaller, cheaper, possibly insufficient
- Higher rank: more expressive, more memory, potentially greater overfitting

A higher rank is not automatically better.

Target modules

Training only attention projections may work for some tasks.

More complex adaptation may require:

- Attention projections
- Output projections
- Feed-forward layers

The selection should be determined experimentally.

11. Evaluation of fine-tuned models

Evaluation should cover more than training loss.

A model can have low validation loss and still produce poor production responses.

Use multiple evaluation layers.

11.1 Task metrics

Classification

Use metrics such as:

- Accuracy
- Precision
- Recall
- F1 score
- Confusion matrix

Example: ticket-routing model.

Input: "The card payment was deducted twice." Target class: PAYMENT_DUPLICATE
--

For rare but important classes, F1 or recall may be more useful than accuracy.

Information extraction

Use:

- Exact match
- Token-level F1
- Field-level accuracy
- Schema-validity rate
- Numeric accuracy

Example evaluation:

Field	Accuracy
Service name	96%
Incident severity	91%
Root cause	78%
Timestamp	94%

Field-level evaluation provides more insight than one overall number.

Summarization

Possible measures include:

- ROUGE-style overlap metrics
- Factual consistency
- Required-fact coverage
- Hallucination rate
- Human preference

Lexical similarity alone is insufficient because several different summaries can all be correct.

Code generation

Possible metrics:

- Unit-test pass rate
- Compilation rate
- Functional correctness
- Security checks
- Static-analysis results

Execution-based evaluation is usually stronger than text similarity.

11.2 Benchmark-style evaluation

A fixed evaluation suite should contain categories such as:

Normal cases
Edge cases
Adversarial inputs
Safety-sensitive requests
Long-context examples
Multilingual examples
Out-of-domain inputs
Formatting requirements
Tool-calling cases

Results should be segmented rather than reported only as one aggregate score.

Example:

Category	Base model	Fine-tuned model
Normal task accuracy	78%	91%
JSON validity	84%	98%
Edge-case accuracy	62%	79%
Out-of-domain refusal	88%	73%
Safety compliance	95%	91%

This example reveals a regression: the fine-tuned model improved task performance but became worse at refusing out-of-domain questions.

11.3 Pairwise preference evaluation

Evaluators compare two responses without necessarily knowing which model generated them.

Prompt
↓
Response A vs Response B
↓
Which is better?

Evaluation criteria may include:

- Correctness
- Relevance
- Completeness
- Conciseness
- Tone
- Groundedness
- Safety
- Actionability

Possible labels:

A is better B is better Tie Both are unacceptable
--

Why pairwise evaluation is useful

Humans often find it easier to choose between two responses than to assign an absolute score from 1 to 10.

Important controls

- Randomize response order
- Hide model identity
- Use multiple evaluators
- Measure inter-rater agreement
- Define a clear rubric
- Resolve disagreements
- Include domain experts for specialized questions

11.4 LLM-as-judge

Another LLM may evaluate outputs using a rubric.

Advantages

- Fast
- Scalable
- Useful during iteration
- Can provide explanations

Risks

- Position bias
- Preference for longer answers
- Self-model preference
- Prompt sensitivity
- Weak domain knowledge
- Inconsistent scoring

Use LLM judges as one signal, not the only source of truth.

A stronger evaluation design combines:

Automated metrics
+
LLM-based evaluation
+
Human evaluation
+
Production monitoring

11.5 Safety evaluation

Test whether fine-tuning weakened the base model's safety behavior.

Evaluate:

- Sensitive data disclosure
- Prompt injection handling
- Harmful instructions
- Unauthorized tool use
- Policy violations
- Fabricated claims
- Excessive confidence
- Refusal quality

A domain adapter can unintentionally make a model more willing to answer unsafe or out-of-scope questions.

11.6 Operational evaluation

A better model is not always a better production system.

Also measure:

- P50 and P95 latency
- Tokens per response
- GPU memory usage
- Throughput
- Cost per request
- Cold-start time
- Adapter-loading time
- Failure rate
- JSON validation rate
- Retry rate

The selected model should optimize overall business value—not only benchmark score.

12. Common pitfalls

12.1 Overfitting

Overfitting occurs when the model memorizes the training examples but performs poorly on new inputs.

Signals

- Training loss continues decreasing
- Validation loss starts increasing
- The model reproduces exact training phrases
- Performance is strong only on familiar templates
- Outputs become less diverse

Mitigations

- Increase dataset diversity
 - Deduplicate examples
 - Reduce training epochs
 - Lower adapter capacity
 - Use early stopping
 - Improve validation splitting
 - Add realistic edge cases
-

12.2 Catastrophic forgetting

Catastrophic forgetting happens when fine-tuning damages capabilities learned during pre-training.

Example:

A model becomes excellent at returning short JSON outputs but loses its ability to:

- Explain answers
- Handle general questions
- Refuse unsafe requests
- Follow unrelated instructions

Causes

- High learning rate
- Too many epochs
- Narrow training distribution
- Full fine-tuning on a small dataset
- Poorly balanced examples

Mitigations

- Use PEFT
- Reduce the learning rate
- Mix some general instruction examples into training
- Evaluate original capabilities
- Use fewer epochs
- Retain the base model for unrelated requests

- Route only relevant tasks to the fine-tuned model
-

12.3 Noisy labels

A model learns the labels it receives, including incorrect ones.

Examples of noisy data:

- Wrong answers
- Incorrect tool calls
- Invalid JSON
- Outdated procedures
- Inconsistent severity labels
- Human annotator disagreements

Fine-tuning can make errors more consistent and more confident.

12.4 Misaligned instruction data

Suppose the production requirement is:

| Ask for confirmation before restarting production services.

But most training examples immediately perform the restart action.

The model will learn behavior that conflicts with the intended safety policy.

The dataset is effectively the specification. Training examples must represent the real desired behavior.

12.5 Training-data leakage

Evaluation results become unreliable when:

- Test examples appear in the training data
- Near-duplicate templates cross splits
- Synthetic test data is generated using training examples
- The same incidents appear in both sets
- Benchmark answers were included in training

Use semantic duplicate detection and grouped splitting.

12.6 Fine-tuning for factual knowledge

Fine-tuning is not a reliable database.

Even after training, the model may:

- Misremember details
- Combine facts incorrectly
- Return outdated information
- Hallucinate confidently

Use RAG or APIs for factual data that must be current, attributable, or updateable.

12.7 Measuring only average quality

An average score may hide serious failures.

A model may achieve 95% accuracy but fail on:

- High-value customers
- Critical incidents
- Minority languages
- Rare safety-sensitive requests
- Very long documents

Evaluation should report sliced metrics.

12.8 Ignoring the chat template

Different models expect different prompt formats.

For example:

```
<system>
...
<user>
...
<assistant>
...
```

The exact special tokens vary by model family.

Training with one format and serving with another can produce major performance degradation.

12.9 Excessive synthetic data

If most examples come from one teacher model, the fine-tuned model may inherit:

- Its tone
- Its hallucinations
- Its biases
- Its limited response diversity

Synthetic data should be filtered, reviewed, and supplemented with real production examples.

13. Senior AI Engineer decision framework

During an interview, explain the decision in this order.

Step 1: Define the failure

Ask:

- Is the model missing facts?
- Is it failing instructions?
- Is the output format inconsistent?

- Is the task too complex for the base model?
- Is the issue limited to certain request categories?
- Are cost and latency the real problems?

Step 2: Establish a baseline

Test:

- Prompt-only
- Few-shot prompting
- RAG
- Structured output
- Larger model
- Smaller model
- Rules or classical ML

Step 3: Confirm the data exists

Determine:

- Number of examples
- Label quality
- Data ownership
- PII and compliance risks
- Production representativeness
- Edge-case coverage

Step 4: Start with PEFT

For most enterprise experiments:

Base model + LoRA or QLoRA + Small controlled dataset

This provides a lower-cost way to test whether training solves the problem.

Step 5: Evaluate broadly

Measure:

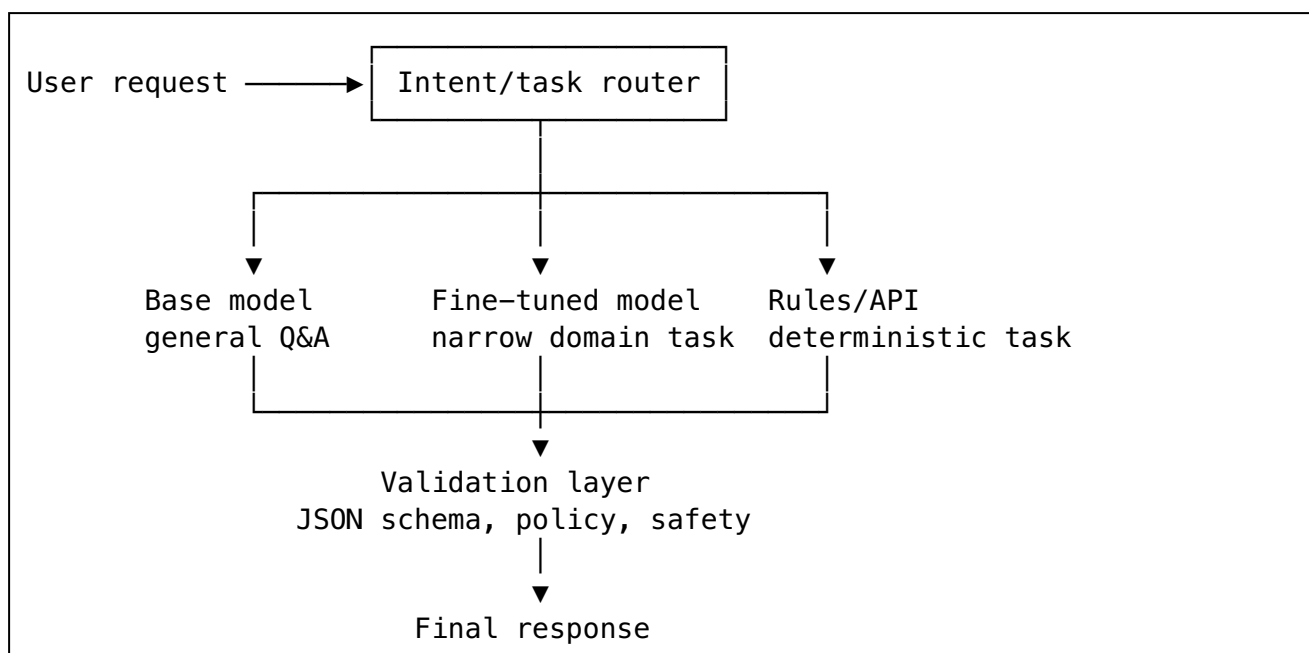
- Task accuracy
- Human preference
- Safety
- General capabilities
- Cost
- Latency
- Production error rate

Step 6: Deploy gradually

Use:

- Offline evaluation
- Shadow traffic
- Canary deployment
- A/B testing
- Rollback support
- Adapter versioning
- Monitoring by use case

14. Example production architecture



A fine-tuned model does not have to replace the general model.

Routing allows each model to handle the tasks for which it is best suited.

15. LoRA/QLoRA interview comparison

Dimension	Full fine-tuning	LoRA	QLoRA
Parameters updated	All or most	Small adapters	Small adapters
Base weights	Trainable	Frozen	Frozen and quantized
GPU memory	Highest	Lower	Lowest
Checkpoint size	Large	Small	Small adapters
Training cost	Highest	Lower	Lower
Maximum adaptation capacity	Highest	High for many tasks	High for many tasks
Forgetting risk	Higher	Lower	Lower

Operational model variants	Full copy per task	Shared base plus adapters	Quantized base plus adapters
Best initial enterprise experiment	Rarely	Often	Often under memory limits

16. Interview Q&A

Q1. What is the difference between pre-training and fine-tuning?

Answer: Pre-training teaches a model general language and representation capabilities using massive, broad datasets, commonly through next-token prediction. Fine-tuning starts with a pretrained model and adapts it to a narrower task, domain, behavior, or style using a smaller targeted dataset.

Q2. When would you choose fine-tuning instead of RAG?

Answer: I would choose fine-tuning when the main problem is behavioral—for example, consistent formatting, domain-specific task execution, style, classification, or tool selection. I would choose RAG when the model needs current, private, frequently changing, or attributable knowledge. Many systems use both: fine-tuning for behavior and RAG for facts.

Q3. What is LoRA?

Answer: LoRA freezes the original model weights and learns small low-rank matrices that approximate the required weight updates. This greatly reduces the number of trainable parameters, GPU memory usage, checkpoint size, and training cost.

Q4. What is the difference between LoRA and QLoRA?

Answer: Both train low-rank adapters while freezing the base model. In QLoRA, the frozen base model is also loaded in low precision, commonly 4-bit, which further reduces memory consumption. QLoRA is especially useful when the full base model cannot fit in higher precision on the available hardware.

Q5. How would you decide whether full fine-tuning is necessary?

Answer: I would first establish prompt, RAG, and PEFT baselines. Full fine-tuning is justified only if the task needs substantial model adaptation, sufficient high-quality data and compute are available, and PEFT does not meet the required quality. I would also evaluate the increased risk of forgetting and the operational cost of maintaining full model copies.

Q6. How do you prepare a fine-tuning dataset?

Answer: I define the target behavior, collect representative production examples, normalize them into the model's expected instruction or chat format, remove PII and secrets, correct labels, deduplicate exact and semantic duplicates, include edge and negative cases, and split the data by logical groups such as incident or customer to prevent leakage.

Q7. How do you evaluate a fine-tuned model?

Answer: I compare it against a clear baseline using task-specific metrics, benchmark-style test suites, pairwise human preference evaluation, safety tests, regression tests for general capabilities, and operational measures such as latency and cost. I also segment results by request category rather than relying only on a single average score.

Q8. What is catastrophic forgetting?

Answer: Catastrophic forgetting occurs when adapting the model to a narrow dataset damages previously learned capabilities. For example, a model may improve at structured extraction but lose general instruction-following or safety behavior. Mitigations include PEFT, lower learning rates, fewer epochs, balanced training examples, and regression testing against the base model.

Q9. Why can noisy instruction data be dangerous?

Answer: Fine-tuning reinforces the patterns in the dataset. Incorrect labels, conflicting procedures, invalid outputs, or unsafe actions can make the model produce those errors more consistently and confidently. Data quality is therefore often more important than raw dataset size.

Q10. Describe a fine-tuning project you would propose for an enterprise system.

Answer: I might fine-tune a smaller model to convert incident logs into a fixed incident schema. I would first compare prompt-only and few-shot baselines, create a grouped train-validation-test split from historical incidents, remove sensitive data, use QLoRA for an initial experiment, and evaluate field-level accuracy, JSON validity, hallucination rate, latency, and cost. The production system would still validate the JSON and route uncertain cases for review.

17. Strong interview summary

A concise Senior AI Engineer answer could be:

Fine-tuning should solve a demonstrated behavioral or task-specific gap, not be the default solution for missing knowledge. I would first establish prompting and RAG baselines, create a representative and leakage-free evaluation set, and verify that sufficient high-quality training data exists. For an initial enterprise experiment, I would normally start with LoRA or QLoRA because they reduce memory, cost, checkpoint size, and forgetting risk. I would evaluate not only task accuracy but also pairwise preference, safety regressions, general capability retention, latency, and cost. Finally, I would deploy the adapter gradually with versioning, monitoring, validation, and rollback support.